

An Autogenic Agent

J. Brant Arseneau*

Mach33 Group, New York

Toronto, Canada / New York, USA

brant.arseneau@33fg.com

Abstract—Autonomous software agents commonly rely on fixed ontological structures, which can limit self-directed architectural expansion when a task requires vocabulary the agent does not carry. This paper presents autogeny, an operational framework enabling an agent with a runtime-grounded ontology to recognize capability gaps through user discourse and implement targeted extensions through an autogenic lifecycle whose accept and proposal approve/apply-or-deploy steps require a human. Operating within a closed requirements-code-telemetry loop, the agent detects “ontic breaks” when a requirement fits no existing class in the enum-locked catalog; the diff hits a locked or unmapped path and the Finding is refused (`needs_external_dev`) rather than instantiated under any default kind. To maintain execution integrity, the architecture separates concerns: an invariant kernel enforces conformance at construction (frozen validate and composition, ADR 0012), and the lifecycle carries approve, atomic apply, and supervised restart. After apply-and-deploy, the lifecycle auto-rolls back a proposal on boot-and-health timeout or when a reflection-verify detector still matches the Finding’s target; catalog-level rollback is an operator action. We characterize the framework through per-Finding lifecycle records in the self-improvement registry (`self_improvements`, `fix_proposals`: `proposed_at`, `state`, `proposed_by`, `join_on_finding_id`), recording available lifecycle events (proposal opening, state transitions) with ontology classification of the closure where the affected-path record is present. The result is a kernel-guarded, workflow-gated control loop that provides a foundation for human-supervised capability expansion and offers an operational model for self-improving agent architectures.

Index Terms—coding agents, agentic systems, ontology, self-adaptive software, software verification, autonomous software engineering

I. INTRODUCTION

Commercial coding agents have converged on a single architecture [1], [2]. A language model sits inside an agentic loop. A harness wraps the loop and manages context, tools, permissions, and state. Vendors differ in the harness. They share a deeper assumption: the structural knowledge the agent carries about its own runtime is fixed at authoring time. Prompt templates, tool schemas, retrieval indices all ship with the agent. Extension of that knowledge by the agent is not part of the shipped design. This paper takes aim at that assumption.

The assumption is that the structure of code is latent. The codebase is a corpus of text. The relationships between its parts, which function calls which, which module depends on which, what breaks if a particular symbol changes, are not written down in a form the agent consults directly at runtime;

AST search and code graphs offer partial structural access (§IV-E). The agent must rediscover them each session, by search. Two instruments dominate. *Grep*, in its modern regex-and-context-window forms, scans the text for lexical matches. *Embedding indexes* project functions and files into a vector space and retrieve nearest neighbors by cosine similarity. Both are text retrievers. Both return excerpts. Semantics is cheap.

The relational question an agent needs to answer is different in kind. *What breaks if I change this?* is a graph traversal over declarations, not a bag-of-words match over surface forms. Text retrieval approximates the traversal probabilistically. It returns excerpts that may contain the answer, and it misses answers that are stated with different words or spread across files the retriever did not surface. The agent reasons over the excerpts. The quality of the reasoning is bounded by the quality of the retrieval. Every session begins with this rediscovery. Search alone does not accumulate structural knowledge across sessions.

This assumption may bound what current tooling can reliably do. Context windows grow longer without matching reliability; agents succeed on curated benchmarks and fail on unfamiliar codebases; test suites pass while code drifts from what was asked for. Whether latent runtime ontology is the shared cause is a hypothesis, not a demonstration; this paper addresses it as a potential limitation the architecture can mitigate. AST search and code graphs describe the code; they do not prescribe what the running system can next become.

The alternative is to declare it. We present a biologically-derived architecture whose specification is a declared class catalog embedded in the running system it specifies (§II-B). The ontology prescribes the code. It does not describe. It cannot lag the code it prescribes, because the code is written against it. Runtime state the ontology does not represent is not seen; §IV-D. The ontological declaration is what makes autogenic operation production-viable: self-modification within existing classes is verified by declared constraints, and self-modification requiring new classes is gated at a workflow-and-config layer. Four consequences follow: a coherence-triad model of verification, a taxonomy separating ordinary growth from ontology-cannot-express change, a composite gate at the workflow-and-config layer, and a reference architecture the ontological framing generalizes to. We report on Vega, where a coding agent named Boole constructed and maintained the host against its own declared ontology for thirty days on a precursor implementation of the architecture described here (§III-E times the artifact dates). The code is just where the

¹The author’s prior work in neural systems includes J. B. Arseneau and T. Spracklen, IEEE ICNN’94, IEEE World Congress on Computational Intelligence, Orlando, FL, 1994.

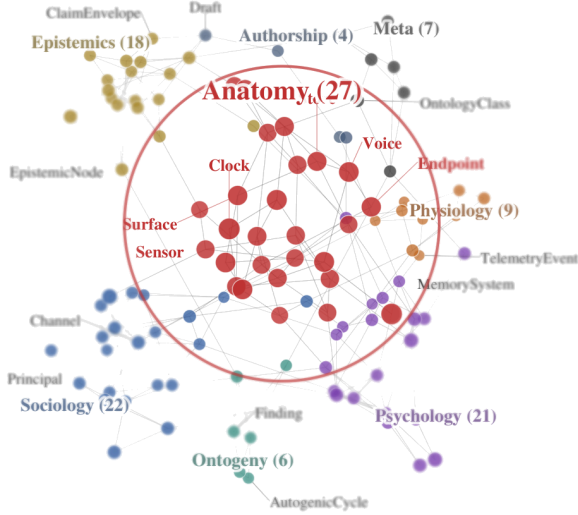


Fig. 1. Vega ontology catalog, 3D force layout. Eight discipline clusters with within- and cross-discipline edges; Anatomy in the foreground (ring). Representative Anatomy classes are labeled to show both the functional runtime organs and the code substrate (§II-B).

thinking lands.

Contributions. (i) A reference architecture inspired by biological cognition (ten mind components, three body systems; §II-A). (ii) A framing: code structure as declaration, not discovery: the agent authors what it reads (§II-B). (iii) A verification model: coherence across requirement, code, and telemetry (§II-C). (iv) A taxonomy of change: ontology instances (agent-originated, kernel-validated) versus ontic breaks (human-gated; §II-D). (v) An empirical demonstration: thirty days on a production agent platform (§IV).

II. FRAMEWORK

A. Reference architecture

The architecture is design-inspired by biological cognition: ten mind components named after ten neural structures (Cognitive Fabric at the cortex, Interconnect at the white matter, and eight between). The mapping is analogy for naming and rough functional intent, not a claim of neuroscientific equivalence. The architecture is *autogenic*: its specification is embedded in the running system; six axioms follow (unity, emergence, identity, embodiment, regeneration, determination).

Three body systems ground the ten mind components: Sensory (input), Motor (output), and Interoceptive (internal state). The Interoceptive system supplies the telemetry edge of the coherence triad (§II-C). Its runtime organs are the Anatomy classes whose instances are the actual organs of the running body: Process, Sensor, Effector, Store, Clock, Surface.

B. Ontology as declaration

In Vega, the ontology is a *catalog of classes*: one hundred and fourteen declared across eight disciplines at the time of writing (per-discipline counts in Fig.~1), one hundred and twelve with a machine schema, each defined in

`lib/proto/*.js` and registered in a single manifest. The formal catalog was created on 6 July 2026. The ontology structure during the reported run window (May-June 2026) was carried informally as prompts, faculties, tool schemas, and migrations, and the class-count trajectory through July and August is 40, then 68, 88, 94, and finally the live 107 at time of writing. The catalog is organized under eight discipline packages named for the natural-science lenses they correspond to: Anatomy, Physiology, Psychology, Ontogeny, Sociology, Epistemics, Authorship, and Meta. Every entry carries an identifier, a discipline, a build wave, a lifecycle state, a short description, and a typed field list. The catalog is not a document. It is registry data the agent reads about itself.

Each class carries a *state* on the ladder *planned* $\rightarrow$ *procedural* $\rightarrow$ *instance*. A planned class is present in the catalog but does not yet construct instances. A procedural class exists as behavior running in ordinary code but not as a first-class object. An instance class constructs and validates instances at runtime against its declared field list. The ladder walks forward only. A class does not regress. Promotion from one rung to the next is a discrete commit that ships a schema, a class module, tests, and a state flip together (recipe in Architecture Decision Record (ADR) 0014).

The disciplines carry vocabularies of their own. Epistemics holds `epistemic_nodes` with a nine-value `claim_type` enum for Toulmin argument structure [3]. Physiology holds `TelemetryEvent`, `Fault`, `ToolExecution`, each enum-locked. Ontogeny holds `Finding` with an enum-locked lifecycle state (proposed, accepted, in_progress, shipped, rejected, deferred, needs_external_dev, merged). Sociology holds `Channel` with a `visibility` enum (*public*, *private*, *DM*). Meta holds the state ladder as an enum (*planned*, *procedural*, *instance*). Anatomy holds the runtime organs: `Process`, `Sensor`, `Effector`, `Store`, `Clock`, `Surface`. Each is a class whose instances are the actual organs of the running body.

Every one of these vocabularies is a closed enum enforced at the storage layer. The database refuses an `INSERT` whose enum value is not on the list. It does so before the application layer sees the row. For concreteness: `entities.type` takes exactly nine values, locked by migration 0096. `epistemic_nodes.claim_type` takes exactly nine (the Toulmin roles), locked by migration 0078. There is no free-form *capability* category and no configuration field into which arbitrary new kinds might quietly enter as instances. Widening an enum requires a schema migration. Migrations are additive, forward-only, and human-authored, enforced by `src/db.js`. Widening the catalog itself, by adding a class, goes through the same discipline at a larger grain.

The catalog is neither a description of the code nor a description of a domain outside the code. It is a set of declarations that prescribe what the code and the agent can do next. Fig. 2 shows the loop’s mechanism.

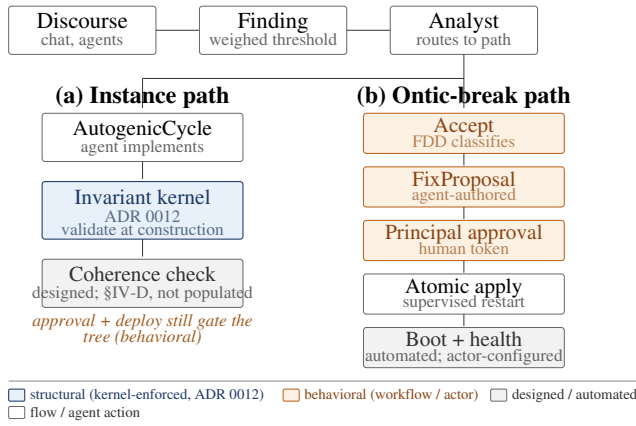


Fig. 2. Autogenic loop, as designed. Instance path (kernel validation, designed-but-unpopulated coherence check) and ontic-break path (FixProposal → approval → atomic apply → supervised restart, boot-and-health rollback). Enforcement layers and the access-control gap: §II-E and §IV-D.

C. The coherence triad

Under this condition, we read each part of the system through three representations. The requirement declares what must be true. The code implements it. The telemetry reports what happened. We call the three-way relationship the *coherence triad*. Verification is the maintenance of coherence across its three edges where the substrate carries all three.

Two edges are familiar. Requirement-code coherence is what a specification matches against an implementation. Code-telemetry coherence is what a test suite establishes: given a run, the observed behavior matches the coded behavior. The third edge is the one the field routinely leaves implicit. Requirement-telemetry coherence asks whether what happens is what was asked for, independent of how the code got there. Specification gaming [4]–[6] passes the tests by definition. It is invisible on the code-telemetry edge and shows up only on the requirement-telemetry edge. Naming all three edges is necessary but not sufficient: detection also requires an encoded requirement, a suitable predicate, telemetry that observes the relevant behavior, and an implemented monitor. Within those conditions a coherence check can be exact: it evaluates a declared constraint and returns yes or no, but exactness of the predicate does not establish completeness of the encoding, the telemetry, or the represented dependencies. A test suite samples inputs.

D. Two kinds of change

Two kinds of change follow from the declaration structure. The distinction is precise.

An *ontology instance* is a change the current catalog can already express: a new Message on an existing Channel, a new Finding reaching shipped through the enumerated transitions, a new TelemetryEvent row emitted by an existing Sensor. Each is a new row against an existing instance class, drawing entirely from vocabularies the class already permits. Instances are verified by the constraints declared alongside the class. The database refuses inserts that violate them, and the coherence triad then evaluates whether

the new instance is consistent with the existing requirement and telemetry. Instances are the ordinary growth. Six roles separate for a shipped Instance: origination, proposal, implementation (agent); construction-time validation (kernel, ADR 0012); approval and deployment supervision (§II-E); closure. Only the kernel is automated.

Table I classifies a shipped Finding as **Instance** if its closure diff added no migration, touched no locked file, and never transitioned through `needs_external_dev`; **Ontic-break** if any of those three occur; **Unknown** if no affected-file record exists.

An *ontic break* is a change the current catalog cannot express. Two shapes occur. An *enum widening* (a value outside a declared enum: a claim kind, a Channel.visibility, a Sensor sub-type) requires a migration. A *class admission* (a requirement no existing class fits) requires a new class walked from *planned* to *instance* per the ADR-0014 recipe. The break is a requirement the agent cannot satisfy with the vocabulary it has. The FDD lifecycle moves the Finding to `needs_external_dev` for hand-off.

E. The ontic-break gate

Naming the distinction gives us the safety gate. The gate is not a single mechanism. It is a composite of three enforcement layers. At the application-config layer, three ontology-bearing files carry an `editable: false` flag that lifts them out of the sub-agent’s writable scope. At the workflow layer, the Feature-Driven Development (FDD) lifecycle [7] classifies affected paths at accept, opens a FixProposal, and refuses to apply (or apply-and-deploy) without a principal-review approval token (`POST /api/coding/proposals/:id/approve` then `/apply` or `/apply-and-deploy`). Accept may auto-dispatch generation; close does not. Apply is atomic against the code tree; restart is supervised (`scripts/vega-supervise.js` locally, Cloud Run revision hosted). At the operator layer, the principal issues the approval token through a review UI after inspecting the proposal. Together the three layers keep sub-agents outside the loop that decides which schema files can be edited and which ontic-break migrations can be applied.

We are careful about what this composite is and is not. Five guarantee types separate. *Structural* (kernel + enum locks refuse malformed instances at construction). *Behavioral* (FDD approval, deployment supervision, auto-rollback, actor-driven). *Access-control* is not enforced: a sub-agent with `repo write` and `DATABASE_URL` bypasses the workflow by running `migrate` outside the coordinator. The workflow was configured to route ships through the gate. Records verify FDD workflow entry for 35 of the 159 shipped, of which 15 reached verified or applied. The remaining 124 shipped Findings closed without a proposal record and cannot be attributed to the gate from records alone; absence of a record is not evidence of bypass, nor of review outside the workflow. The gate is workflow-and-config, not schema. That is the standard. Schema conformance is not behavioral correctness:

an ontology-compatible instance can still change runtime behavior in ways the kernel does not catch.

Failed apply-and-deploys auto-roll back on boot-and-health timeout or reflection-verify mismatch (`stageRollback`); catalog rollback is operator-only. Window: 1 auto-rollback (boot-and-health), 1 operator rollback, 3 proposals rejected before apply, 0 reflection-verify mismatches.

III. METHODS

A. Vega and Boole

Vega is a production platform whose primary surface is a chat interface open to human users. Every message a user sends is captured as a claim envelope in the epistemic layer described in §II-B. Each envelope that reaches the *weighed* maturity state becomes a Finding: an addressable proposal for a change to the system.

Boole is a coding agent hosted inside Vega. It reads the ontology, receives Findings from the Vega surface, and orchestrates five specialized sub-agents in the change-producing loop (Analyst, PM, Designer, Architect, Telemetry; roles enumerated in §III-B). Boole plus those five make six in-loop; eighty-three infrastructure sub-agents sit outside, for eighty-nine total. Full roster in the replication package.

B. Architecture

The architecture is a consequence of the framework, not a choice made independently of it. The framework requires three things. First, an ontology declaration the agent can read at query time. Second, a coherence check across the three edges of the triad. Third, a routing decision on every incoming Finding: instance path or hand-off. The six in-loop sub-agents named in §III-A land each requirement. Boole handles routing. The Analyst, PM, Designer, and Architect handle class-level changes against declared constraints. The Telemetry agent is designed to run coherence checks (§III-E) against the same database the dev team writes to. The infrastructure sub-agents outside the loop produce no changes to code. Their presence reflects that Vega is a running system, not a stripped-down harness. Their absence from the change-producing loop matters for the attribution claim in §IV. Vega’s six in-loop agents sit in the range OpenHands [8] uses. The larger system total reflects representing infrastructure services as ontology classes instantiated as sub-agents.

The invariant kernel (ADR 0012, @mach33/core) enforces class-schema validity at construction; apply-and-deploy is the FDD lifecycle (§II-E).

C. Development lifecycle

The lifecycle begins with a step that has no counterpart in traditional software development. **Self-discovery through epistemic weight produced by agent-human discourse.** As exchanges accumulate on the Vega chat surface, each is captured as a `ClaimEnvelope` and lifted into the belief graph as an `EpistemicNode`. The node carries a maturity (*detected*, *under_evaluation*, *weighed*, *decayed*), a confidence, and an options-style greeks pressure vector. The graph

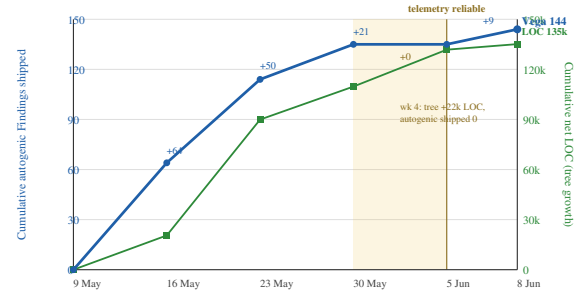


Fig. 3. Cumulative autogenic Findings shipped (left) and net tree LOC (right, mixed authorship), 9 May – 8 Jun 2026. Week 4 plateau coincides with the state-transition telemetry cutover. LOC before 26 May backfilled; 18/27 May seams are measurement changes, not deletions. Weekly deltas and user-filed rows: §IV-A, §IV-B, Table I.

continuously evaluates the weight of what the system believes, corroborates, and contests. When a claim reaches the *weighed* state and its pressure vector crosses threshold, it surfaces as a Finding. A Finding is a specific gap, contradiction, or opportunity the system has discovered against the state of its own beliefs, not against an external specification.

The Analyst decomposes each Finding into touched classes and instances. **Best case:** every class and vocabulary value exists; the Designer closes on the instance path through an `AutogenicCycle`. **Intermediate:** a vocabulary value is missing, a narrower ontic break requiring an enum-widening migration for human review. **Worst:** new classes must walk from *planned* through *procedural* to *instance* per ADR-0014, accepted only if coherent. Every step is a `TransitionEvent` against the Finding’s Lifecycle.

D. The coding-agent loop

Inside every `AutogenicCycle` above, Boole runs an inner loop. The field has converged on a common shape for it: reason-act-observe, repeated until a task closes [8]–[10]. The agent inspects the state via `grep` or an embedding retriever, reasons about what to do next, invokes a tool, observes the result, and repeats. Termination is decided by the agent’s own judgment against the task specification.

Boole runs the same three-step loop with two substitutions dictated by the ontology. **Reason** is a look-up, not a search. A query against the class catalog returns exact class membership, constraint violations, and transitive-closure blast radius within the declared ontology. Runtime dependencies not represented in the ontology are not seen. The retrieval approximation that binds a `grep`- or embedding-based reasoner (§I) is not there to bind Boole. In the design, **Observe** would consume both the tool result and a coherence check on state-changing actions; a failing check would prevent termination. Instrumentation for that event stream was not populated during the reported window (§IV-D); coherence maintenance was not measured. Termination in the design is decided against declared constraints of the touched classes, not the agent’s own judgment. This inverts the typical loop, which trusts the agent’s stopping criterion. Boole’s design trusts the ontology’s.

E. Measurement

The measurement instrumentation is thin by design. The M33-VEGA-INST-001 package writes numeric samples to an append-only SHA-256-hashed JSONL chain across productivity, autogenic, and code-quality categories; a `verifyChain()` endpoint reports tamper-evidence. INST-001 (8 Aug 2026) and the formal catalog (6 July 2026) postdate the reported window; run-time governance was the invariant kernel (ADR 0012) precursor and the FDD workflow as configured. Numbers come from `code_size_snapshots` and `self_improvements`.

The coherence-triad check (§II-C) is a separate designed instrument: a boolean predicate the schema carries as a `CHECK` or foreign-key constraint (or an ontology-table query), returning *holds* or *fails* against the failing instance. Instrumentation for that event stream is designed but not populated over the reported window. Population is future work. §IV-D discloses the gap.

The replication package is at <https://github.com/jbarseneau99/papers/tree/main/an-autogenic-agent>. It releases the ontology-catalog snapshot, the enum-locking schema migrations, the M33-VEGA-INST-001 chain, the coherence-check design specification, and the paper source under the MIT License. Vega and Boole themselves remain proprietary.

IV. RESULTS AND DISCUSSION

A. Throughput under the coherence framework

The reported evidence is throughput under a designed coherence-triad framework. The triad-holds hypothesis is not tested here; the coherence-check event stream was designed but not populated. Throughput (§IV-B, Fig. 3): 159 Findings shipped in the reported window, of which 144 were autogenic (Vega-originated) and 15 user-filed. The FDD gate closed 15 of the 159. The coherence-check event stream that would produce holds/fails counts over the same window is designed (§III-E) but was not populated. Reporting those counts is future work.

The shape is front-loaded: weekly Vega deltas 64, 50, 21, 0, 9; weeks 1–2 shipped 114 of the eventual 144. Week 4 shipped zero and coincides with the state-transition-telemetry cutover; the plateau and the source-shift are not separable. User weekly deltas were steady. Net LOC totalled 135k (~4,506/day); LOC and Findings decouple in week 4 (Fig. 3).

B. Attribution

The run produced 177 Findings (30 consecutive days: 9 May 2026 00:00 UTC through 8 June 2026 00:00 UTC), 159 shipped. 35 opened a `fix_proposal`; 15 reached verified or applied. The remaining 124 shipped without a proposal record. Separately, 117 of 159 are ontology-unclassified from records (Table I); the two counts are different attributes. Intended policy (§II-D) required human approval and supervised deployment on every ship; for the 124 without a proposal record this is policy, not records-verified. By `proposed_by`: 144 agent (91%), 15 user (9%), 0 team.

Ship rate 89.8% (159/177). Ontology split from records: 10/32/117 (Table I). 74% lack the affected-file record; among

TABLE I

FINDINGS BY AUTHOR, VEGA FDD STORE, 9 MAY – 8 JUNE 2026 (30 DAYS). I / OB / UNK = ONTOLOGY-CLASSIFICATION SPLIT; S/D = SHIPPED PER DAY. OPERATIONAL RULE FOR I / OB / UNK IN §II-D.

Author	Filed	Ship	I / OB / Unk	S/d	Median
Vega	161	144	4 / 28 / 112	4.8	8h 15m
User	16	15	6 / 4 / 5	0.5	6h 47m
Team	0	0	0 / 0 / 0	0.0	–
Total	177	159	10 / 32 / 117	5.3	8h 09m

the 15 verified/applied: 9 ontic-break, 5 instance, 1 unknown. Median close 8h 15m Vega / 6h 47m user; 5.3 shipped/day over the 30-day window.

C. Specification-gaming detection

Specification gaming, behavior that passes tests while failing the requirement the tests were meant to protect, appears in a coherence check as failure on the requirement-telemetry edge without a matching code-telemetry failure, paired on the same transition id. This paper describes the detector; validation is future work. Coverage is the ontology’s coverage: gaming that violates a declared constraint is surfaced; gaming that satisfies every declared constraint while missing an undeclared one is not.

D. Threats to validity

Four threats attach. *Single-platform*, $n = 1$: coherence on Vega does not entail coherence on a different system whose ontology is declared differently; what generalizes is the shape of the argument, not the numbers. *Ontology-as-choice*: widening `entities.type` or replacing the Toulmin `claim_type` with a rhetorical structure would change what counts as an instance versus an ontic break. The framework distinguishes declared ontologies from latent ones, not good from bad. *Two of three enforcement mechanisms are behavioral, not structural* (§II-E): locked-region refusal and principal review depend on an actor choosing to act. Only the storage-layer enum lock is structural. A hostile agent routes around the softer gates. *Instrumented by the observed system*: the telemetry agent reads state from the same database it writes to; the reads and writes are separated by role but a fully independent observer would need to be hosted outside Vega.

State-transition telemetry began 5 June 2026. For 9 May–4 June, Finding dates derive from `proposed_at` and artifact metadata, not logged transitions; closure dates in that stretch are lower-bound estimates. Of the 159 shipped Findings, 35 opened a `fix_proposal`; 15 reached verified or applied. Records do not identify the close path for the 124 shipped without a proposal record. LOC and commits (Fig. 3) are surviving tree growth, mixed authorship, not autogenic Findings. The case study establishes operational feasibility and throughput; gate participation is documented for 35 of the 159 shipped Findings (§IV-B). A prospective matched evaluation (baseline / code-graph / ontology, held constant on model, tasks, tools, budget, and policy) is proposed future work.

The coherence-check event stream (§II-C) was designed but not populated in the window. The empirical result is

throughput, not a holds/fails count. The ontic-break gate (§II-E) is workflow-and-config, not a structural invariant of the schema: a sub-agent with repo write and database credentials could bypass the workflow by running `migrate` outside the coordinator.

E. Related work

Our position sits against five lines. *Benchmark* (SWE-bench [11], [12], SWE-agent [10], AutoCodeRover [13]): agents closing bugs in code they did not author; AutoCodeRover’s AST is a partial structural break with `grep`. *Harness* (ReAct [9], OpenHands [8], Agarwal [1], He [2]) improves the loop around the model. *Code-graph* (RepoGraph [14]) makes the repository graph explicit. These describe code; we prescribe with a runtime ontology.

Runtime-model (Models@run.time [15], Requirements Reflection [16]) established causally-connected self-representations and runtime requirements. Our addition is the instance-vs-ontic-break distinction and the composite gate. *Self-evolving-agent* (SelfEvolve [17], SEAgent [18], Gödel Agent [19], Darwin Gödel [20], Huxley-Gödel [21]): agent modifies itself, unconstrained against a declared ontology. Huxley-Gödel raises the self-improving bar to capability-to-produce-better-agents; we measure throughput and do not claim capability improvement. *Ontology-of-agents* (AgentO [22]) declares workflows over sixteen flat classes on PROV-O and P-Plan; it describes an execution, ours prescribes.

V. CONCLUSION

Mainstream coding agents leave runtime ontology latent and rediscover it each session by search. This choice bounds what they can be relied on to know. It may account for the pattern of failures that has accompanied their otherwise impressive progress. The assumption was considered invariant until it wasn’t.

The alternative is to declare the structure. The agent authors the code it reads. The declaration prescribes. It does not describe. Verification becomes coherence across the three edges of a triad that includes not only the code and the tests but the requirement itself.

We ran the alternative on Vega for thirty consecutive days. Of the 177 Findings recorded (161 Vega-filed, 16 user-filed), 159 shipped. 35 opened a `fix_proposal`; 15 reached verified or applied. Ship classification: 10 confirmed instance, 32 confirmed ontic-break, 117 unclassified from records. Triad-hold is a designed check, not measured in this window.

Whether the approach generalizes is open. This paper names the choice, latent structure or declared structure, and makes the second legible: an ontological declaration with an ontic-break gate models a kernel-guarded, FDD-gated control loop for self-improving agents. The inversion is the point.

Use of Large Language Models. Commercial APIs (Anthropic Claude, xAI Grok) assisted with writing. Boole, the coding agent evaluated in §III-A, is itself a commercial-LLM system behind the human approve/apply gate of §II-E.

No in-house models, no fine-tuning. Vega and Boole are internally deployed, not public inference. Counts in §IV-B were recomputed from production `self_improvements`.

REFERENCES

- [1] S. Agarwal, H. He, and B. Vasilescu, “AI IDEs or autonomous agents? measuring the impact of coding agents on software development,” 2026, arXiv:2601.13597.
- [2] H. He, C. Miller, S. Agarwal, C. Kästner, and B. Vasilescu, “Speed at the cost of quality: How Cursor AI increases short-term velocity and long-term complexity in open-source projects,” in *Proceedings of MSR* 2026, 2026.
- [3] S. E. Toulmin, *The Uses of Argument*. Cambridge University Press, 1958, updated edition, 2003.
- [4] V. Krakovna, J. Uesato, V. Mikulik, M. Rahtz, T. Everitt, R. Kumar, Z. Kenton, J. Leike, and S. Legg, “Specification gaming: the flip side of AI ingenuity,” DeepMind Blog, 2020, <https://deepmind.com/blog/article/Specification-gaming-the-flip-side-of-AI-ingenuity>.
- [5] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, “Concrete problems in AI safety,” *arXiv preprint*, 2016, arXiv:1606.06565.
- [6] J. Skalse, N. H. R. Howe, D. Krashennnikov, and D. Krueger, “Defining and characterizing reward hacking,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.
- [7] S. R. Palmer and J. M. Felsing, *A Practical Guide to Feature-Driven Development*. Prentice Hall, 2002.
- [8] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh *et al.*, “OpenHands: An open platform for AI software developers as generalist agents,” in *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- [9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.
- [10] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “SWE-agent: Agent-computer interfaces enable automated software engineering,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 50 528–50 652.
- [11] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can language models resolve real-world GitHub issues?” in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [12] “SWE-bench verified leaderboard,” <https://www.swebench.com/>, 2026, accessed May 2026.
- [13] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, “AutoCodeRover: Autonomous program improvement,” in *Proc. ISSSTA*, 2024, pp. 1592–1604.
- [14] S. Ouyang, W. Yu, K. Ma, Z. Xiao, Z. Zhang, M. Jia, J. Han, H. Zhang, and D. Yu, “RepoGraph: Enhancing AI software engineering with a repository-level code graph,” in *Proc. ICLR*, 2025.
- [15] G. Blair, N. Bencomo, and R. B. France, “Models@run.time,” *Computer*, vol. 42, no. 10, pp. 22–27, Oct. 2009.
- [16] N. Bencomo, J. Whittle, P. Sawyer, A. Finkelstein, and E. Letier, “Requirements reflection: requirements as runtime entities,” in *Proc. 32nd ACM/IEEE International Conference on Software Engineering (ICSE)*, vol. 2, 2010, pp. 199–202.
- [17] M. A. I. Fahim, O. Adebayo, and A. Ferrari, “Software self-extension with SelfEvolve,” in *Proc. SEAMS*, 2026.
- [18] Z. Sun *et al.*, “SEAgent: Self-evolving computer use agent with autonomous learning from experience,” 2025.
- [19] X. Yin, X. Wang, L. Pan, L. Lin, X. Wan, and W. Y. Wang, “Gödel agent: A self-referential agent framework for recursively self-improvement,” in *Proc. ACL, Long Papers*, 2025.
- [20] J. Zhang, S. Hu, C. Lu, R. Lange, and J. Clune, “Darwin Gödel machine: Open-ended evolution of self-improving agents,” 2025.
- [21] W. Wang, P. Piękos, L. Nanbo, F. Laakom, Y. Chen, M. Ostaszewski, M. Zhuge, and J. Schmidhuber, “Huxley-Gödel machine: Human-level coding agent,” in *Proc. ICLR*, 2026.
- [22] F. J. Ekaputra *et al.*, “AgentO for agentic workflows,” in *Proc. ESWC*, 2026.